Laboratório SENTINELA: do circuito digital ao IP Core — Relatório

Ezequias Carlos Ferreira da Silva e Marcelo Gomes Barbosa · Logisim-evolution 5.0.0 · github.com/marcelobns/sentinela-ip-core

1. Entrega e verificação

O `sentinela.circ` contém `alu8`, `regs2`, `fir4`, `intertravamento`, `sentinela_core` (a integração `fir4` + `intertravamento`) e `main` (botões, LEDs e displays). Um modelo de referência ciclo a ciclo em Python (`ref/sentinela_ref.py`) confere as contas do roteiro e gera os cinco arquivos de vetores: 1097 verificações, todas aprovadas; o `main` passa em 138/138 numa bancada com pinos no lugar de Clock e Buttons. Como o clear do Register do Logisim é assíncrono, o reset síncrono entra pelo D (mux com 0) e pelo enable.

2. Etapa 1 — alu8 e regs2

O carry da SUB é o **empréstimo** (`b_out` do Subtractor: 1 quando a < b), a única semântica que satisfaz os casos 3 e 4; em ARM a convenção é a inversa (C = não-empréstimo). A SHR é só fiação: y = {0, a[7:1]} e carry = a[0]. O `regs2` tem vetor próprio (`test_regs2.txt`), porque um Test Vector roda sobre um único circuito [VERIFICAR: o roteiro pede o caso 7 em `test_alu8.txt`].

Reflexiva. Em portas NAND2 equivalentes (estimativa grosseira, somador completo ≈ 8 portas; WESTE; HARRIS, 2011), a `alu8` tem ≈ 210 portas (somador ≈ 64, subtrator ≈ 70, mux de saída ≈ 56, AND, zero e carry ≈ 20). Somador e subtrator são ~60% da área, e a cada mudança de a ou b as quatro unidades comutam, então a maior parte da potência dinâmica vai para resultados descartados. Um somador/subtrator compartilhado, a + (b XOR sub) + sub (PATTERSON; HENNESSY, 2021), elimina ≈ 50 portas (~25%); a avaliação guardada, que isola os operandos das unidades não selecionadas (TIWARI; MALIK; ASHAR, 1995), corta a comutação inútil, com um pouco de lógica a mais no caminho. A alternativa ao Shifter é a adotada: deslocar 1 bit fixo é fiação, de custo e atraso nulos; um barrel shifter gastaria ≈ 55 portas para um deslocamento constante.

3. Etapa 2 — fir4

Por que cabe em 8 bits: como Σh = 128, y é uma média ponderada das entradas e nunca passa da maior delas; no pior caso o acumulador chega a 255 · 128 = 32640 < 2^{16}, e 32640 >> 7 = 255. **ciclos = 5:** a borda do `start` conta 1 e cada uma das 4 bordas de MAC soma 1; `y_out` é registrado na borda do último tap, junto com a entrada em DONE, para estar válido enquanto `done = 1`, que é quando a FSM o amostra. **Soma de 16 bits:** três `alu8` em modo ADD (parte baixa, parte alta e o carry da baixa), porque a `alu8` não tem carry de entrada e encadear duas exigiria mudar sua interface; mantém o reaproveitamento (opção a), ao custo de três ALUs em série no caminho crítico. **Caso 5 [VERIFICAR]:** com 0x64, 0x64, 0xFF, 0x64 (mais antiga primeiro), o pico cai em x[n−1], de peso 48: (16·100 + 48·255 + 48·100 + 16·100) >> 7 = 20240 >> 7 = 158 = **0x9E** (79 °C), e não ≤ 0x80; ainda assim, 0x9E < T_VENT, e o pico não liga a ventilação (caso 10).

Reflexiva. Na mesma métrica, o multiplicador 8×8 em array tem ≈ 500 portas e o `fir4` inteiro ≈ 1,8 mil. Com quatro multiplicadores e uma árvore de somadores, a área iria a ≈ 2,8 mil (razão ≈ 1,5; 4× no multiplicador) e a latência cairia de 5 para 2 ciclos (speedup 2,5) ou 1 (5). Mas o tempo por amostra inclui a aquisição: com relógio de 1 MHz (5 µs de cálculo) e 100 µs de conversão A/D, a fração acelerada é f ≈ 0,048 e, pela Lei de Amdahl (AMDAHL, 1967; HENNESSY; PATTERSON; KOZYRAKIS, 2025), S = 1 / ((1 − f) + f / 2,5) ≈ 1,03 — 3% de ganho por 50% a mais de área; mesmo com cálculo instantâneo, S ≤ 1,05, limite que vale também para multinúcleos (HILL; MARTY, 2008). Num motor amostrado a 1 Hz, f ≈ $5 \cdot 10^{-6}$. O paralelismo só compensa quando o cálculo domina, como nos aceleradores de domínio específico. E, com coeficientes fixos, 16x e 48x = 32x + 16x são deslocamentos e uma soma, sem multiplicador (DEMPSTER; MACLEOD, 1995), ao preço de perder a ROM parametrizável.

4. Etapa 3 — intertravamento

Máquina de Moore; transições só com `amostra_ok = 1`; o `rst` síncrono leva a NORMAL. Saídas da tabela: v = vent, a = alarme, c = corte, f = falha_sensor. `sf` = (temp > T_FAIXA_MAX) OU (run ≥ N_CONGELADO), em que run conta as amostras idênticas seguidas, incluindo a atual, e satura em 15. Contando igualdades, como sugere a dica, 8

amostras iguais dariam 7 e a falha sairia na 9ª, contra o caso 8 [VERIFICAR]. ALARME volta a VENT com temp < T_CORTE: o diagrama não mostra essa saída, mas o caso 4 a exige [VERIFICAR]. Com `sf`, o `ack` é ignorado.

Estado	Código	v a c f	Próximo estado (condições em ordem de prioridade)
NORMAL	000	0 0 0 0	`sf` → FALHA; `temp ≥ T_VENT` → VENT; senão NORMAL
VENT	001	1 0 0 0	`sf` → FALHA; `temp ≥ T_CORTE` → ALARME; `temp ≥ T_VENT` → VENT; senão NORMAL
ALARME	010	1 1 0 0	`sf` → FALHA; `temp ≥ T_CORTE` → CORTE; senão VENT
CORTE	011	1 1 1 0	`sf` → FALHA; `ack` → NORMAL; senão CORTE
FALHA	100	1 1 1 1	`sf` → FALHA; `ack` → NORMAL; senão FALHA
inválido	101–111	1 1 1 1	FALHA na borda seguinte, sem esperar `amostra_ok`

FALHA deve cortar o gerador? Para o motor, sim: é falhar para o estado seguro (fail-safe; AVIZIENIS et al., 2004), e o gerador já queimou com uma leitura congelada. Mas o corte deixa a escola sem energia, e o detector de congelamento — a falha stuck-at, ou CONSTANT, da literatura de sensores (NI et al., 2009; SHARMA; GOLUBCHIK; GOVINDAN, 2010) — tem falso positivo: com temperatura estável e sem ruído (x_in = 0x80 constante), a saída filtrada fica igual a partir da 4ª amostra e o SENTINELA corta um gerador saudável na 11ª; com ruído de ±1 LSB, não. O filtro, que existe para remover ruído, torna repetições mais prováveis. Melhorias: aplicar o detector à leitura bruta, e não à filtrada, e, em FALHA, manter ventilação e alarme e cortar só se não houver `ack` num prazo.

Reflexiva. Implementamos o controle hardwired. O microprogramado (bônus, não implementado) usaria uma ROM endereçada por {estado, sf, ge_c, ge_v, ack} — 7 bits, 128 palavras de 3 bits —, com `amostra_ok` como enable do registrador de estado. Um estado MANUTENCAO é mais fácil na ROM: há códigos livres (101–111) e basta regenerar o conteúdo a partir da tabela, sem mexer na fiação. No hardwired, é preciso refazer as equações e as portas. Em latência, o hardwired tende a vencer: 2 a 3 níveis de porta após os comparadores, contra decodificar o endereço e ler a ROM; aqui, nanossegundos são irrelevantes diante do período de amostragem, e nas duas o corte ocorre na borda seguinte ao done. É o compromisso clássico: o controle hardwired, como a unidade de controle que Patterson e Hennessy (2021) derivam de uma tabela-verdade, é mais rápido e compacto quando o controle é simples; o microprogramado, proposto por Wilkes (1951), ganha em flexibilidade quando é complexo — por isso é comum combinar os dois: hardwired nas instruções comuns e microcódigo nas complexas.

5. Ponte para o PTT

Este laboratório prepara a trilha **PTT4**, no SenseBridge, biblioteca em Rust para ESP32 que integra o DHT11 e a Open-Meteo com validação, operação degradada e recuperação. O embrião é o `intertravamento`: a detecção de sensor fora de faixa corresponde à validação local, a FSM aos estados de operação e o rearme por `ack` à transição de recuperação; `ciclos` e o modelo com vetores antecipam a medição de latência e os cenários C1 a C5. O laboratório também mudou o SenseBridge: a validação de faixa aceitava um DHT11 travado num valor plausível, e ele ganhou um detector de leitura congelada. Como a resolução de 1 °C torna repetições legítimas comuns (até 64 ciclos seguidos nas capturas), o detector é opcional, usa um limiar longo (360 ciclos, 30 min, ainda a validar em hardware) e guarda memória própria, fora da FSM. Ele marca a leitura como suspeita sem alterar o estado e, se configurado, reinicia o sensor no máximo uma vez por limiar. Onde o SENTINELA vai para o estado seguro e espera o rearme, o SenseBridge, que não aciona nada perigoso, degrada (AVIZIENIS et al., 2004): tenta recuperar a fonte e registra se a leitura mudou depois do reinício, o que distingue um sensor travado de um ambiente estável.

Referências

AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. In: AFIPS SPRING JOINT COMPUTER CONFERENCE, 1967, Atlantic City. **Proceedings** [...]. New York: ACM, 1967. p. 483–485. DOI: 10.1145/1465482.1465560.

AVIZIENIS, A. et al. Basic concepts and taxonomy of dependable and secure computing. **IEEE Transactions on Dependable and Secure Computing**, v. 1, n. 1, p. 11–33, 2004. DOI: 10.1109/TDSC.2004.2.

DEMPSTER, A. G.; MACLEOD, M. D. Use of minimum-adder multiplier blocks in FIR digital filters. **IEEE Transactions on Circuits and Systems II**, v. 42, n. 9, p. 569–577, 1995. DOI: 10.1109/82.466647.

HENNESSY, J. L.; PATTERSON, D. A.; KOZYRAKIS, C. **Computer architecture**: a quantitative approach. 7. ed. Cambridge: Morgan Kaufmann, 2025.

HILL, M. D.; MARTY, M. R. Amdahl's law in the multicore era. **Computer**, v. 41, n. 7, p. 33–38, 2008. DOI: 10.1109/MC.2008.209.

NI, K. et al. Sensor network data fault types. **ACM Transactions on Sensor Networks**, v. 5, n. 3, art. 25, 2009. DOI: 10.1145/1525856.1525863.

PATTERSON, D. A.; HENNESSY, J. L. **Computer organization and design RISC-V edition**: the hardware software interface. 2. ed. Cambridge: Morgan Kaufmann, 2021.

SHARMA, A. B.; GOLUBCHIK, L.; GOVINDAN, R. Sensor faults: detection methods and prevalence in real-world datasets. **ACM Transactions on Sensor Networks**, v. 6, n. 3, art. 23, 2010. DOI: 10.1145/1754414.1754419.

TIWARI, V.; MALIK, S.; ASHAR, P. Guarded evaluation: pushing power management to logic synthesis/design. In: INTERNATIONAL SYMPOSIUM ON LOW POWER DESIGN, 1995, Dana Point. **Proceedings** [...]. New York: ACM, 1995. p. 221–226. DOI: 10.1145/224081.224120.

UFRR. **Laboratório — Projeto SENTINELA**: do circuito digital ao IP Core. Boa Vista: PPCA/UFRR, 2026. Roteiro de laboratório.

WESTE, N. H. E.; HARRIS, D. M. **CMOS VLSI design**: a circuits and systems perspective. 4. ed. Boston: Addison-Wesley, 2011.

WILKES, M. V. The best way to design an automatic calculating machine [1951]. In: LEWIS, H. R. (ed.). **Ideas that created the future**: classic papers of computer science. Cambridge: MIT Press, 2021. p. 165–168. DOI: 10.7551/mitpress/12274.003.0017.